

DTT Connection Status

Documentation | 17-05-2022





Table of Contents

1. Get started quickly	3
2. Introduction	4
3. Set-Up	5
4. Editor	6
5. API	9
6. Known Limitations	12
7. Support and feedback	13

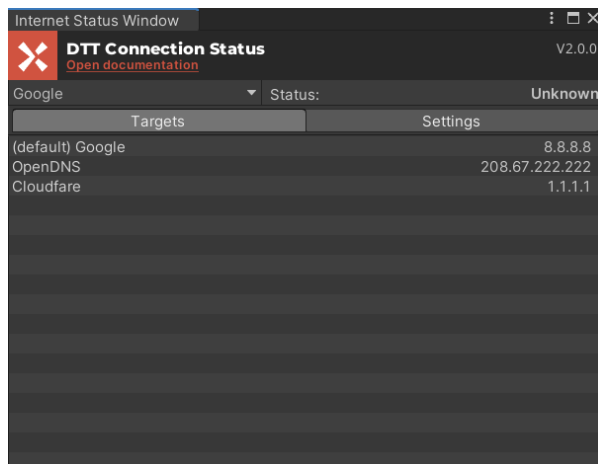


1. Get started quickly

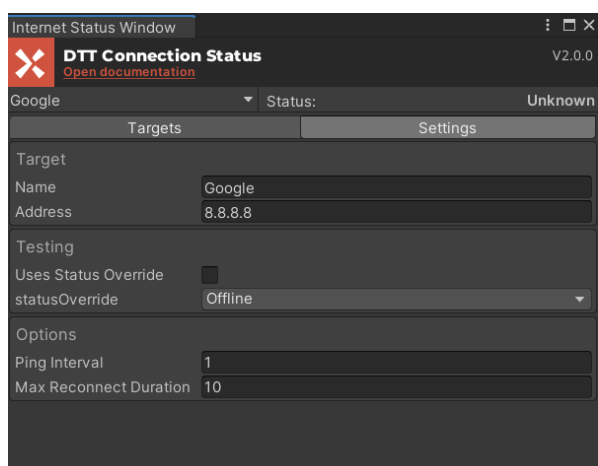
This is a summary to help you get started quickly. More details are provided below.

To get started, import the **DTT.Networking.ConnectionStatus** namespace into your script file. This will allow you to use the **InternetStatusManager** to interact with internet status targets. Each target has three events: **OnOnline**, **OnOffline** and **StatusUpdate**. You can use these events to respond to changes in your connection to the target.

If you want to edit the default target settings or add new custom ones you can open the **InternetStatus** window by navigating to: Tools > DTT > ConnectionStatus > Window.



In the **Targets** section you get an overview of all your targets with their respective address.



In the **Settings** section you are able to configure the settings of a selected internet status target. Some of these settings will only be relevant during playmode which is discussed more in depth in the **Editor** section of the documentation.

You now know everything you need to start using the Connection Status package. For more details on the asset's extensive features, please keep reading.



2. Introduction

DTT Connection Status is a Unity package that allows you to check the current connection status for your project and other custom internet status targets. It gives you the ability to subscribe methods to events for when a target goes offline, is reconnecting, and finally returns online again. Using the package's editor window you can customize and manage your internet status targets and how their connection status is determined. This allows for an easy way to test and manage the connection statuses in your game.



3. Set-Up

To start using the asset, start by importing the **DTT.Networking.ConnectionStatus** namespace into your script file like in the following example:

```
using DTT.Networking.ConnectionStatus;
using UnityEngine;

public class CustomMonobehaviour : MonoBehaviour
{
}
```

You can now start referencing the **ConnectionStatusManager** class and get target info like so:

```
private void Start()
{
    // The default target can be used to check your internet connection and
    // respond to changes in connection status.
    InternetStatusManager.DefaultTarget.StatusUpdate += RespondToStatusUpdate;

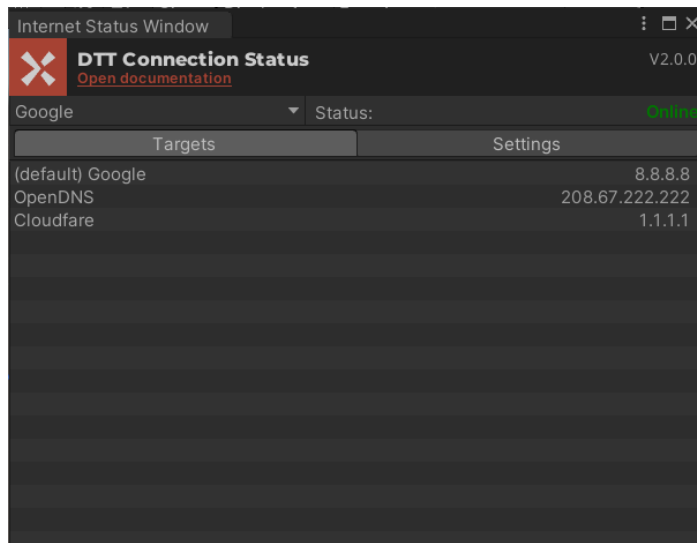
    // Retrieve a custom target you added in the editor window to check its status.
    InternetStatusTarget openDNS = InternetStatusManager.GetTarget("OpenDNS");
    Debug.Log(openDNS.CurrentStatus);

    // You can also retrieve a target based on the target's address.
    InternetStatusTarget cloudFare = InternetStatusManager.GetTarget("1.1.1.1");
    Debug.Log(cloudFare.CurrentStatus);
}
```

Want to edit your default target settings or add a new custom one? You can do that by opening the window using Tools > DTT > Connection Status > Window.

4. Editor

The connection status window is made to work both in the editor and during play mode. As you start play mode, the status of the currently selected target in the window will now update with the current status of the target.

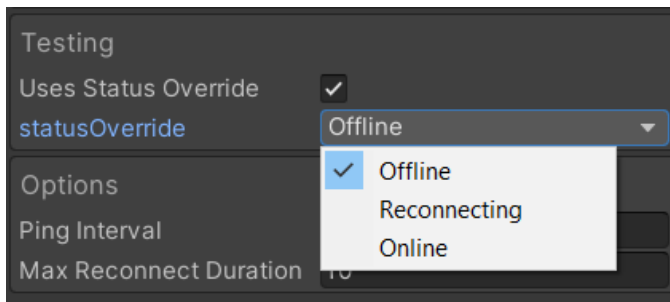


Double-clicking a target in the 'Targets' view will switch your selected target. You can also click the target at the top left of the screen to open a popup to select the new target you want to see the status of.

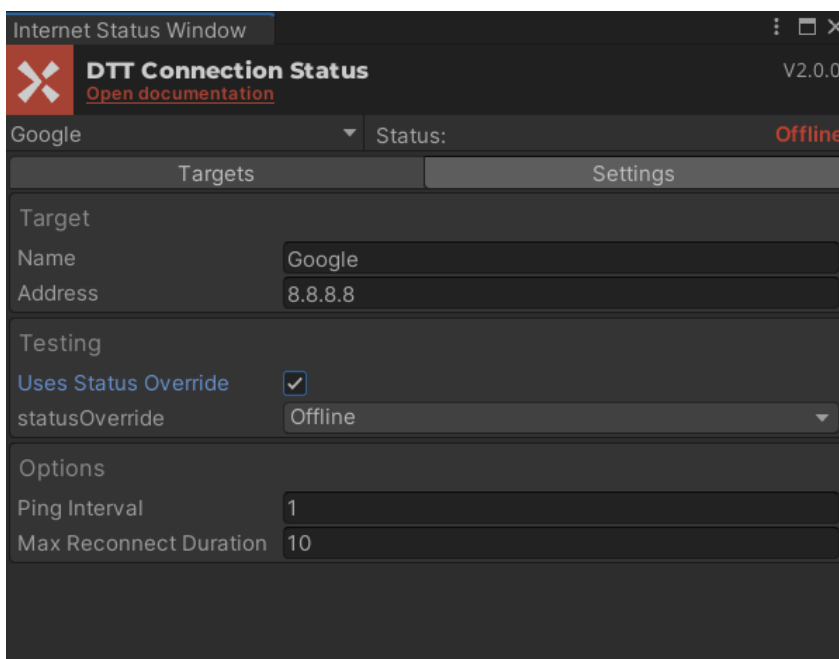


Testing

The testing section contains the **Uses Status Override** toggle that allows you to override when your target sends events and shows the status of your target's connection. If you've used **Connection Status** as your main connection checking tool throughout your game, you can use the testing window to test your game in different connection states.

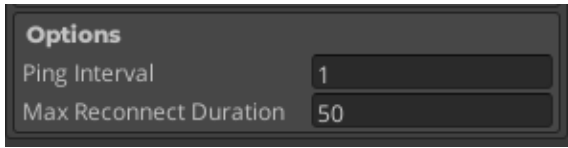


Once toggled, you can set any of the three possible `InternetStatus` values, which are **Online**, **Offline**, **Reconnecting**. When you change these values, the corresponding events are triggered after the ping interval finishes. So if you set the override status to **Online**, the **OnOnline** event will be invoked, just like it would when a user actually successfully connects. Also the **StatusUpdate** will be invoked each time you change the status in this window.





Options



Options	
Ping Interval	1
Max Reconnect Duration	50

The options field allows you to change the **ping interval** and **maximum reconnect duration**.

- The **ping interval** allows you to change the frequency with which the given address is pinged to check if the user still has a connection. The default value is '1', which means the address is pinged every second.
- The **maximum reconnect duration** is used when calling the **AttemptReconnect** method. It serves as the maximum waiting time until the ping is deemed unsuccessful.



5. API

Events

```
// Subscribe to the OnOnline event to get a callback when we get online.  
InternetStatusManager.DefaultTarget.OnOnline += MyOnOnlineMethod;  
  
// Subscribe to the OnOffline event to get a callback when we get offline.  
InternetStatusManager.DefaultTarget.OnOffline += MyOfflineMethod;
```

By referencing **InternetStatusManager.DefaultTarget**, you can subscribe to events on the default target.

OnOnline is invoked once the user regains connection, while **OnOffline** is invoked once the user loses connection. You can use these events to start certain behaviour for when the user's connection changes.



```
private void Awake()
{
    // Retrieve a custom target from the Manager.
    InternetStatusTarget openDNS = InternetStatusManager.GetTarget("OpenDNS");

    // Use the 'On' method to execute code once after a given internet status is being set.
    // The 'On' method will execute the code immediately if the current status is already active.
    openDNS.On(InternetStatus.OFFLINE, () =>
    {
        /* Do something once, after going/being offline */
    });

    // Do something after the internet status has updated.
    openDNS.StatusUpdate += MyStateChangeBehaviour;
}

private void OnDestroy()
{
    // Retrieve a custom target from the Manager.
    InternetStatusTarget openDNS = InternetStatusManager.GetTarget("OpenDNS");

    // Stop listening to status updates.
    openDNS.StatusUpdate -= MyStateChangeBehaviour;
};
```

Another way to invoke certain behaviour after the connection changes, is by using the **StatusUpdate** event. This event is invoked once the user either goes online or offline, but also when the user starts attempting to reconnect or when the connection status is unknown.



Attempting to reconnect

```
InternetStatusTarget openDNS = InternetStatusManager.GetTarget("OpenDNS");

// Attempt a reconnect using the maximum duration for set for our custom target.
openDNS.AttemptReconnect();

// Attempt a reconnect and do something with the result.
openDNS.AttemptReconnect((succeeded) =>
{
    /* Do something with the result of the attempted reconnect. */
});

// Attempt a reconnect and do something with the result after a custom minimum duration.
openDNS.AttemptReconnect(2f,(succeeded) =>
{
    /* Do something with the result of the attempted reconnect after 2 seconds. */
});
```

Another method you can use on a target is **AttemptReconnect**. This method can take 2 arguments at most. The first parameter defines the minimum wait time until a response is returned and the second parameter is a callback that gets invoked once the reconnect attempt is finished. If you leave the minimum wait time empty, the **max reconnect duration** specified in the editor window will be used instead.



6. Known Limitations

There are currently no limitations.



7. Support and feedback

If you have any questions regarding the use of this asset, we are happy to help you out.

Always feel free to contact us at:

unity-support@d-tt.nl

(We typically respond within 1-2 business days)

We are actively developing this asset, with many future updates and extensions already planned. We are eager to include feedback from our users in future updates, be they 'quality of life' improvements, new features, bug fixes or anything else that can help you improve your experience with this asset. You can reach us at the email above.

Reviews and ratings are very much appreciated as they help us raise awareness and to improve our assets.

DTT stands for Doing Things Together

DTT is an app, web and game development agency based in the centre of Amsterdam. Established in 2010, DTT has over a decade of experience in mobile, game, and web based technology.

Our game department primarily works in Unity where we put significant emphasis on the development of internal packages, allowing us to efficiently reuse code between projects. To support the Unity community, we are publishing a selection of our internal packages on the Asset Store, including this one.

More information about DTT (including our clients, projects and vacancies) can be found here:

<https://www.d-tt.nl/en/>